

Python

Summary Session 18



Преподаватель

Портрет

Имя Фамилия

Текущая должность

Количество лет опыта

Какой у Вас опыт - ключевые кейсы

Самые яркие проекты

Дополнительная информация по вашему усмотрению

Корпоративный e-mail

Социальные сети (по желанию)

Важно

- 

Камера должна быть включена на протяжении всего занятия
- 

В течение занятия вопросы задавать в чате или когда преподаватель спрашивает, есть ли у Вас вопросы
- 

Вести себя уважительно и этично по отношению к остальным участникам занятия
- 

Организационные вопросы по обучению решаются с кураторами, а не на тематических занятиях
- 

Во время занятия будут интерактивные задания, будьте готовы включить камеру или демонстрацию экрана по просьбе преподавателя



ПОВТОРЕНИЕ ИЗУЧЕННОГО



Экспресс-опрос



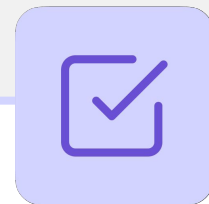
Что такое HTTP?



Протокол HTTP (Hypertext Transfer Protocol)

Этот протокол является основным для обмена данными в Интернете. Он определяет структуру и правила взаимодействия между клиентом и сервером

важно



HTTP используется для передачи гипертекстовых документов, таких как веб-страницы, изображения и другие ресурсы



URL (Uniform Resource Locator)

Это адрес ресурса в сети. Он определяет местонахождение ресурса и используется для доступа к нему

Экспресс-опрос



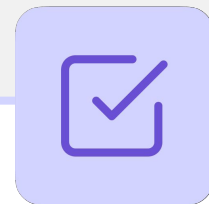
Что такое DNS?



DNS (Domain Name System)

Система, предоставляющая механизм преобразования удобочитаемых имен доменов в IP-адреса, которые используются для связи с серверами

важно



Любое наше взаимодействие с ресурсами в сети Интернет – это отправка HTTP-запроса и получение HTTP-ответа

Экспресс-опрос

?

Перечислите основные типы запросов

Основные типы запросов

GET

получить данные

POST

отправить данные на сервер,

PUT

обновить данные

DELETE

удалить данные

Взаимодействие с ресурсами в интернет



HTTP-запрос

- метод запроса
- URL ресурса
- заголовки запроса
- тело запроса (для POST-запросов)

HTTP-ответ

- код состояния
- заголовки ответа
- тело ответа



Модуль requests

Одна из наиболее популярных библиотек для работы с HTTP-запросами и HTTP-ответами

Модуль requests. Установка

```
pip install requests
```


Get-запрос. Пример

```
import requests

response = requests.get("https://example.com")
print(response.status_code) # Выводит код состояния ответа
print(response.headers)    # Выводит заголовки ответа
print(response.text)       # Выводит данные, полученные от сервера
```

Экспресс-опрос

?

Назовите полезные свойства и методы объекта `response`

Полезные свойства и методы объекта `response`



`.status_code` – код ответа сервера



`.headers` – словарь с заголовком ответа



`.text` – ответ как строка



`.json()` – если сервер вернул JSON, этот метод превратит его в Python-словарь или список

Аргумент params



Аргумент `params` используется для передачи определенных параметров, от которых будет зависеть ответ сервера, с `get`-запросом

Пример использования params

```
# Пример GET-запроса с параметрами
url = 'https://httpbin.org/get' # Сервис для тестирования HTTP-запросов
params = {'course': 'Python', 'lang': 'ru'}

response = requests.get(url, params=params)

# Вывод результата
if response.status_code == 200:
    print(response.json()) # Тело ответа в формате JSON
else:
    print(f"Status code: {response.status_code}")
```

Применение декоратора



Параметры будут переданы серверу в URL-адресе после символа ?

Ответ сервера в программе будет аналогичен результату набора в адресной строке

<https://httpbin.org/get?course=Python&lang=ru>

Запросы POST, PUT, DELETE

```
# Пример POST-запроса с данными
url = 'https://httpbin.org/post' # Сервис для тестирования HTTP-запросов
data = {
    'name': 'example',
    'job': 'tester'
}

response = requests.post(url, json=data)

# Вывод результата
if response.status_code == 200:
    print(response.json()) # Тело ответа в формате JSON
else:
    print(f"Status code: {response.status_code}")
```

Экспресс-опрос



Что такое API?



API (Application Programming Interface)

Интерфейс для программного взаимодействия с веб-сервисом. В общем случае API – это набор определенных правил и протоколов, которые позволяют различным приложениям взаимодействовать между собой

Для работы с API необходимо



Найти и изучить **документацию API**



Из документации узнать:

- URL для обращения к API (так называемый endpoint),
- обязательные параметры (например, ключ),
- формат ответа (чаще всего JSON).



Адаптировать обращение к API под Python с requests

Изучение документации



<https://random-data-api.com/documentation>

Список ресурсов сервиса

Available Resources

More resources are being added every week

Users - </users>

Address - </addresses>

Banks - </banks>

Appliances - </appliances>

Beers - </beers>

Blood Types - /blood_types

Credit Cards - /credit_cards

Формат обращения к ресурсам

Parameters

Base URI: <https://random-data-api.com/api/v2/>

Use [size=number](#) to fetch an array of any resource.

Use [response_type=json](#) to render JSON or [response_type=xml](#) to render XML.

Example #1: https://random-data-api.com/api/v2/users?size=2&is_xml=true

Example #2: <https://random-data-api.com/api/v2/cannabis>

Пример: Обращение к API ресурса users

```
response = requests.get('https://random-data-api.com/api/v2/users?size=1')
res=response.json()
for key, value in res.items():
    print(f"{key}: {value}")
```

Пример: Более сложный запрос к API

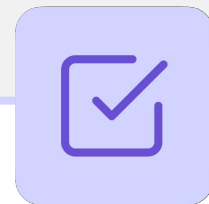
```
params = {'response_type' : 'json', 'size' : 10}
response = requests.get('https://random-data-api.com/api/v2/users',
params=params)
res=response.json()
for person in res:
    print(f"{person['last_name']:15}    {person['first_name']:15}
{person['email']}")
```

Экспресс-опрос



В чем особенность подключения к сайтам, требующим регистрации и подписки?

Важно



Доступ к сайтам, требующим регистрации и подписки, осуществляется по специальному уникальному API-ключу.

Работа с weatherapi.com



Зарегистрируйтесь на сайте weatherapi.com



Подтвердите регистрацию



Войдите в созданный аккаунт и найдите вверху страницы строку с API Key



Изучите документацию API на странице по ссылке <https://www.weatherapi.com/docs/>

Пример: запрос текущей погоды

```
API_KEY = "сюда нужно вставить ваш API-ключ"
city = "Berlin"
params = {'key': API_KEY, 'q': city}
url = f"http://api.weatherapi.com/v1/current.json"

response = requests.get(url, params)
data = response.json()

for key, value in data.items():
    print(f"{key}: {value}")
```

Пример: полезная информация из json

```
print(f"Город: {data['location']['name']}")
print(f"Время: {data['location']['localtime']}")
print(f"Температура: {data['current']['temp_c']} C")
print(f"Ощущается как: {data['current']['feelslike_c']} C")
print(f"Влажность: {data['current']['humidity']} %")
```

Запрос-ответ



Пояснения

- Можно обращаться к веб-сайтам из скрипта python
- Чаще всего полученный ответ – это HTML-документ

Пример

```
import requests

response = requests.get("https://example.com")
print(response.text)
```

Экспресс-опрос



Что такое HTML?



HTML (HyperText Markup Language)

Это язык разметки, с помощью которого создаются и структурируются веб-страницы. Он указывает браузеру, что отображать (заголовки, текст, изображения, таблицы и т.д.) и как это организовано на странице

Инструменты для работы сайта

HTML

Разметка страницы - шапка,
заголовки, абзацы и т.п.

CSS

оформление – цвета, шрифты,
отступы

JavaScript

интерактивность – нажатия,
анимации и т.п.

HTML-теги



Пояснения

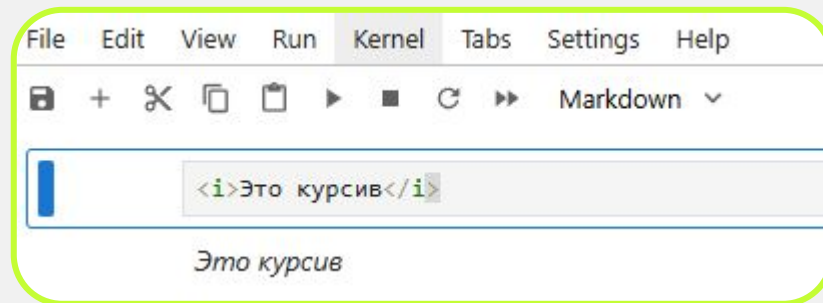
- HTML-документ состоит из **тегов**, которые обрамляют каждый элемент страницы и говорят, как этот элемент должен выглядеть
- Большинство тегов имеют **открывающую и закрывающую форму**

Пример

`<i>Это курсив</i>`

Просмотр в Jupyter Notebook

В Jupyter Notebook можно использовать HTML-теги и отображать результат их применения в ячейках типа Markdown



HTML-теги



Пояснения

- Существуют самозакрывающиеся теги

Пример

```

```

Структура HTML-документа



Каждый HTML-документ начинается с `<!DOCTYPE html>`, затем идут основные блоки

Экспресс-опрос

?

Какие HTML-теги вы помните ?

Структура HTML-документа



Пример

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8">
    <title>Заголовок страницы</title>
  </head>
  <body>
    <!-- Вся видимая часть страницы -->
  </body>
</html>
```

Пояснения

- `<html>` — корневой элемент HTML-документа
- `<head>` — невидимая часть документа, содержит метаданные, стили, скрипты и заголовок страницы
- `<title>` — задаёт заголовок страницы
- `<meta>` — содержит метаданные
- `<body>` — основное содержимое страницы

Основные теги HTML. Текст



`<h1>` – `<h6>` – заголовки, где `<h1>` – самый крупный



`<p>` – абзац (параграф)



`<strong>`, `<b>` – жирный текст



`<em>`, `<i>` – курсив



`<br>` – перенос строки



`<hr>` – горизонтальная линия

Основные теги HTML. Списки



`<ul>` — маркированный список (unordered list)



`<ol>` — нумерованный список (ordered list)



`<li>` — элемент списка (list item)

Основные теги HTML. Ссылки и изображения



`<a href="ссылка">` — гиперссылка



`` — изображение

Пример

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <title>Моя первая страница</title>
</head>
<body>
  <h1>Добро пожаловать!</h1>
```

<p>Это пример простой HTML-страницы. Здесь вы увидите разные теги.</p>

```
<h2>Список языков:</h2>
<ul>
  <li>HTML</li>
  <li>CSS</li>
  <li>JavaScript</li>
</ul>
```

1

```
<h2>Изображение:</h2>

```

```
<h2>Ссылка:</h2>
<p><a href="https://example.com">Перейти на
внешний сайт</a></p>
```

```
</body>
</html>
```

2

Основные теги HTML. Таблицы



`<table>` — таблица



`<tr>` — строка таблицы



`<th>` — заголовочная ячейка (bold + по центру)



`<td>` — обычная ячейка

Пример

```
<table>
  <tr>
    <th>Имя</th>
    <th>Возраст</th>
  </tr>
  <tr>
    <td>Анна</td>
    <td>22</td>
  </tr>
  <tr>
    <td>Олег</td>
    <td>30</td>
  </tr>
</table>
```

Экспресс-опрос



Что такое парсинг?



Парсинг

Это процесс автоматического извлечения данных с веб-страниц. Например, можно спарсить заголовки новостей, курсы валют, расписание фильмов, цены в интернет-магазине и т.д.

Применение парсинга



Анализ данных



Мониторинг цен и новостей



Сбор контента из публичных источников



BeautifulSoup

Это библиотека Python для удобного извлечения данных из HTML и XML. Она превращает HTML в дерево тегов, по которому можно легко искать нужные элементы. Часто используется вместе с библиотекой `requests`, которая помогает получать HTML-код веб-страницы.

BeautifulSoup



Установка

```
pip install beautifulsoup4
```

Пример

```
import requests
from bs4 import BeautifulSoup

url = 'https://example.com'
response = requests.get(url)

if response.status_code == 200:
    soup = BeautifulSoup(response.text, 'html.parser')
    title = soup.title.text
    print('Заголовок страницы:', title)
else:
    print('Ошибка при запросе:', response.status_code)
```

Экспресс-опрос

?

Какие методы поиска вы помните?

Основные методы поиска



`find(tag, attrs)` — находит **первый** тег с указанными условиями



`find_all(tag, attrs)` — находит **все** подходящие теги



`.text` или `.get_text()` — извлекает текст из тега



`.get('атрибут')` — извлекает значение атрибута, например href.

Пример 1

```
print(soup.find('h1'))  
print(soup.find('h1').text)
```

```
# Первый <h1>
```

```
# Текст первого <h1>
```

Пример 2

```
print(soup.find('a'))
print(soup.find('a').text)
print(soup.find('a').get('href'))
print(soup.find('a')['href'])
```

```
# Первый <a>
# Текст первого <a>
# Ссылка из первого <a>
# Альтернативный способ
```

Пример 3

```
list_p = soup.find_all('p')           # Все <p>
print(list_p)

for tag in list_p:
    print(tag.text)
```

Экспресс-опрос

?

Что подходит для парсинга таблиц?

Обработка таблиц



Пояснение

Парсить таблицы можно
с помощью
BeautifulSoup

Пример

```
table = soup.find('table')
rows = table.find_all('tr')

for row in rows:
    cols = row.find_all('td')
    data = [col.text.strip() for col in cols]
    print(data)
```


Пример 1

```
url = "https://the-internet.herokuapp.com/tables"
response = requests.get(url)

if response.status_code == 200:
    soup = BeautifulSoup(response.text, "html.parser")

    # Находим первую таблицу
    table = soup.find("table", id="table1")

    # Извлекаем заголовки
    headers = [th.text.strip() for th in table.find_all("th")]
    print("Заголовки:", headers)

    # Извлекаем строки данных
    rows = table.find_all("tr")[1:] # Пропускаем заголовок
    for row in rows:
        cols = [td.text.strip() for td in row.find_all("td")]
        print(cols)
else:
    print("Ошибка запроса:", response.status_code)
```

Работа со страницей

<https://the-internet.herokuapp.com/tables>

Пример 2

```

table = soup.find("table", id="table2")

headers = [th.text.strip() for th in table.find_all("th")]

# Парсинг в словари
data = []
rows = table.find_all("tr")[1:] # Пропускаем строку заголовков
for row in rows:
    cols = [td.text.strip() for td in row.find_all("td")]
    row_dict = dict(zip(headers, cols))
    data.append(row_dict)

# Вывод результата
for entry in data:
    print(entry)

```

Табличные данные удобнее иметь в форме списка словарей



ВОПРОСЫ





ЗАДАНИЕ





Ответьте на вопрос

1.1 Какую библиотеку в Python чаще всего используют для работы с HTTP-запросами?

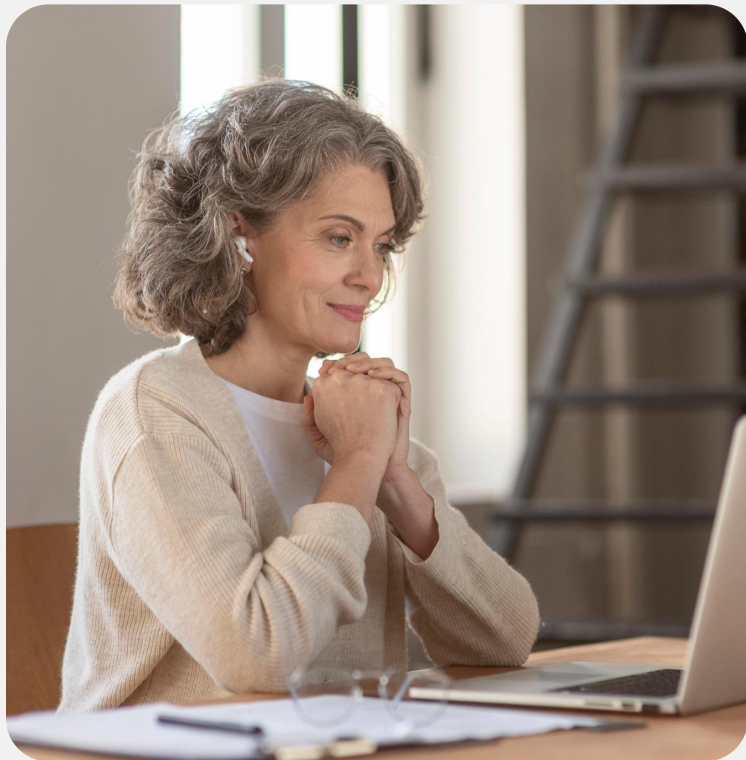
- a. `urllib`
- b. `json`
- c. `requests`
- d. `flask`



Ответьте на вопрос

1.1 Какую библиотеку в Python чаще всего используют для работы с HTTP-запросами?

- a. urllib
- b. json
- c. requests
- d. flask



Ответьте на вопрос

1.2 В каком случае нужно использовать POST-запрос вместо GET?

- a. Когда запрашиваются изображения
- b. Когда нужно получить данные
- c. Когда нужно отправить данные на сервер
- d. Когда требуется авторизация



Ответьте на вопрос

1.2 В каком случае нужно использовать POST-запрос вместо GET?

- a. Когда запрашиваются изображения
- b. Когда нужно получить данные
- c. Когда нужно отправить данные на сервер**
- d. Когда требуется авторизация



Ответьте на вопрос

1.3 Что вернёт метод `.json()` у объекта `response`, если сервер не вернул JSON?

- a. None
- b. Ошибку
- c. Пустую строку
- d. Массив



Ответьте на вопрос

1.3 Что вернёт метод `.json()` у объекта `response`, если сервер не вернул JSON?

- a. None
- b. Ошибку**
- c. Пустую строку
- d. Массив



Ответьте на вопрос

1.4 Какой из вариантов является корректным способом передачи параметров в GET-запросе?

- a. `requests.get(url, data=params)`
- b. `requests.get(url, params=params)`
- c. `requests.get(url, json=params)`
- d. `requests.get(url, headers=params)`



Ответьте на вопрос

1.4 Какой из вариантов является корректным способом передачи параметров в GET-запросе?

- a. `requests.get(url, data=params)`
- b. `requests.get(url, params=params)`
- c. `requests.get(url, json=params)`
- d. `requests.get(url, headers=params)`



Ответьте на вопрос

1.5 Какой код статуса обычно означает «не найдено»?

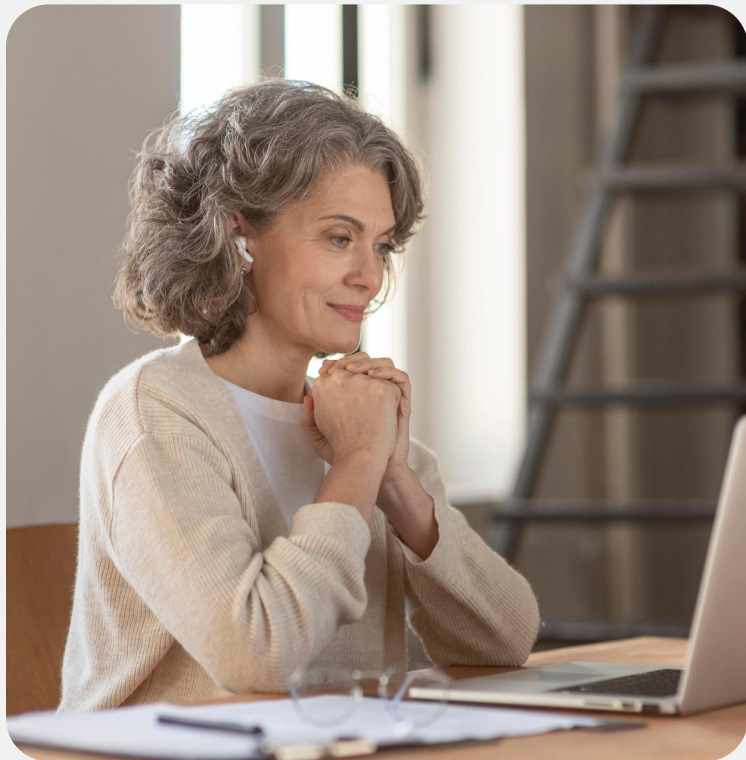
- a. 200
- b. 403
- c. 404
- d. 500



Ответьте на вопрос

1.5 Какой код статуса обычно означает «не найдено»?

- a. 200
- b. 403
- c. 404
- d. 500



Ответьте на вопрос

1.6 Найдите ошибку в коде:

```
response =
requests.get("https://api.site.com/data",
params={'user': 'admin'})
data = response.json
print(data['name'])
```

- a. Метод .json нужно вызвать с круглыми скобками
- b. Параметры передаются в URL после знака ?
- c. Нужно было использовать метод post
- d. Ошибка в адресе URL



Ответьте на вопрос

1.6 Найдите ошибку в коде:

```
response =
requests.get("https://api.site.com/data",
params={'user': 'admin'})
data = response.json
print(data['name'])
```

- a. Метод `.json` нужно вызвать с круглыми скобками
- b. Параметры передаются в URL после знака ?
- c. Нужно было использовать метод `post`
- d. Ошибка в адресе URL



Ответьте на вопрос

1.7 Какую информацию чаще всего содержит документация к API?

- a. Примеры на Python
- b. Список параметров запросов, URL-эндпоинты и формат ответа
- c. Ссылки на GitHub
- d. Список всех пользователей сервиса



Ответьте на вопрос

1.7 Какую информацию чаще всего содержит документация к API?

- a. Примеры на Python
- b. Список параметров запросов, URL-эндпоинты и формат ответа**
- c. Ссылки на GitHub
- d. Список всех пользователей сервиса



Ответьте на вопрос

1.8 Что такое endpoint в контексте API?

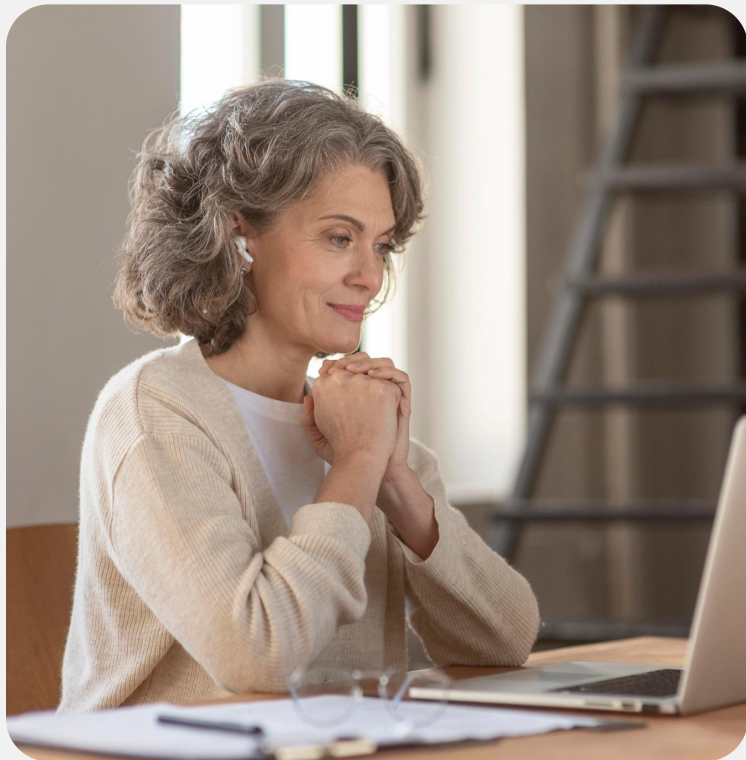
- a. Заголовок HTTP-запроса
- b. Ключ доступа к API
- c. Конкретный URL или маршрут, к которому обращаются для получения данных
- d. Папка с результатами API



Ответьте на вопрос

1.8 Что такое endpoint в контексте API?

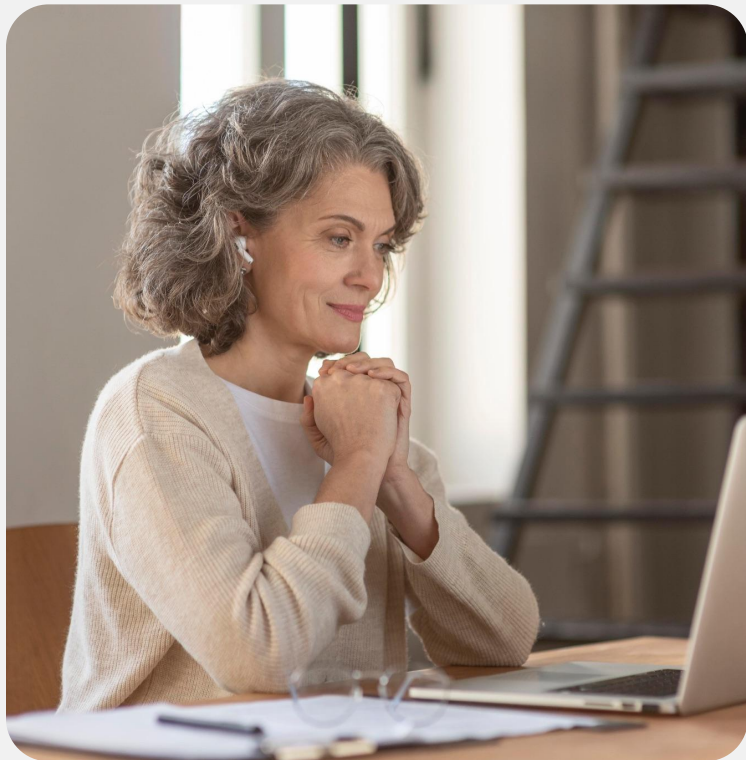
- a. Заголовок HTTP-запроса
- b. Ключ доступа к API
- c. Конкретный URL или маршрут, к которому обращаются для получения данных
- d. Папка с результатами API



Ответьте на вопрос

1.9 **Что** **такое** **API** **key?**

- a. Адрес сервера
- b. Название функции API
- c. Уникальный идентификатор для доступа к API
- d. Метод защиты от вирусов



Ответьте на вопрос

1.9 Что такое API key?

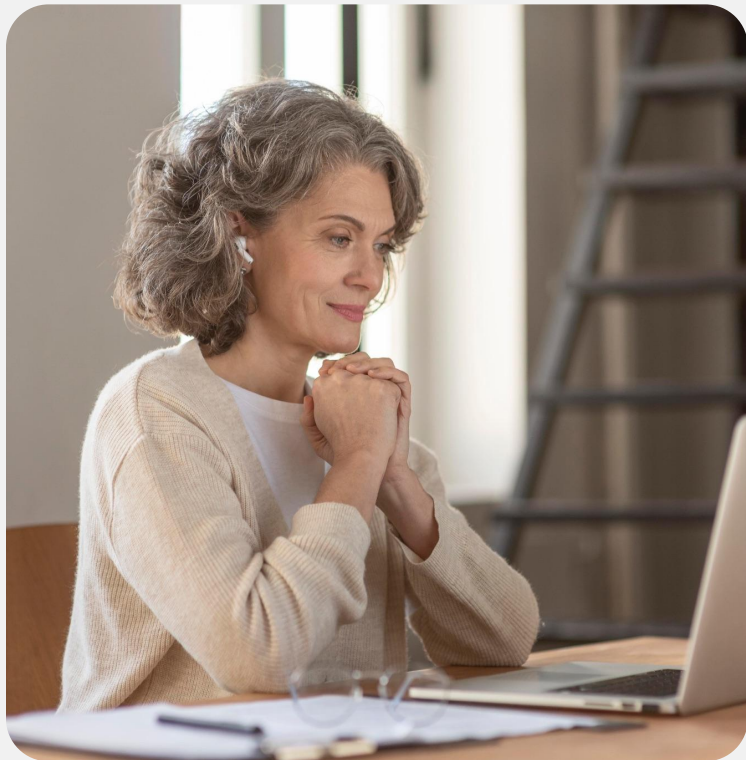
- a. Адрес сервера
- b. Название функции API
- c. Уникальный идентификатор для доступа к API
- d. Метод защиты от вирусов



Ответьте на вопрос

1.10 Что произойдёт, если не указать обязательный параметр при обращении к API?

- a. API автоматически заполнит его
- b. Вернётся случайный результат
- c. Запрос проигнорируется
- d. API вернёт ошибку (обычно 400 или 422)



Ответьте на вопрос

1.10 Что произойдёт, если не указать обязательный параметр при обращении к API?

- a. API автоматически заполнит его
- b. Вернётся случайный результат
- c. Запрос проигнорируется
- d. API вернёт ошибку (обычно 400 или 422)**



ВОПРОСЫ





Ответьте на вопрос



2.1 Что такое HTML?

- a. Язык программирования для создания динамических веб-приложений
- b. Язык разметки для структурирования и отображения содержимого веб-страниц
- c. Стилизованный язык для оформления веб-страниц
- d. База данных для хранения информации веб-сайтов



Ответьте на вопрос

2.1 Что такое HTML?

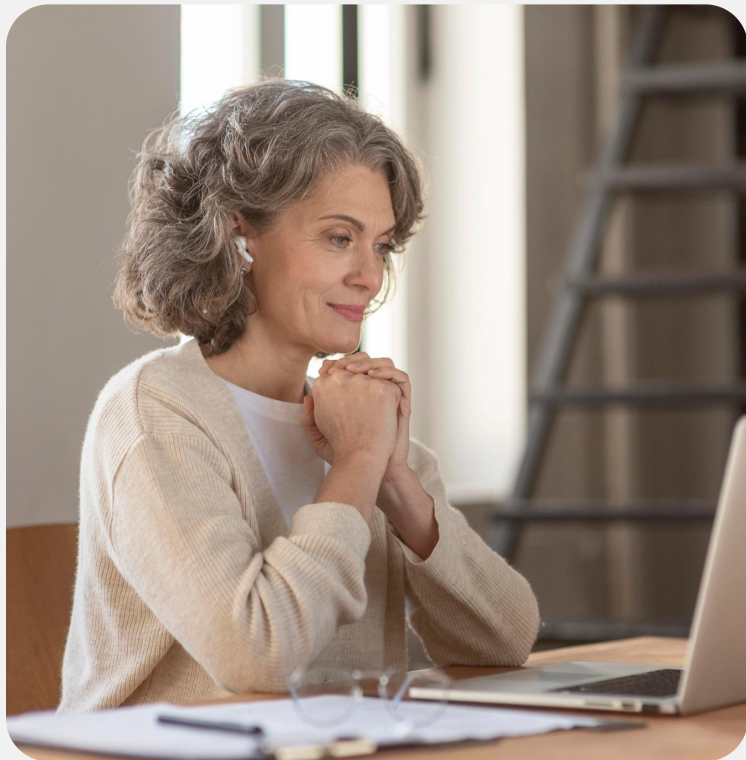
- a. Язык программирования для создания динамических веб-приложений
- b. Язык разметки для структурирования и отображения содержимого веб-страниц
- c. Стилизованный язык для оформления веб-страниц
- d. База данных для хранения информации веб-сайтов



Ответьте на вопрос

2.2 Какой тег используется для создания нумерованного списка в HTML?

- a. `<ul>`
- b. `<ol>`
- c. `<li>`
- d. `<list>`



Ответьте на вопрос

2.2 Какой тег используется для создания нумерованного списка в HTML?

- a. `<ul>`
- b. `<ol>`
- c. `<li>`
- d. `<list>`



Ответьте на вопрос

2.3 Какой тег используется для вставки изображения в HTML?

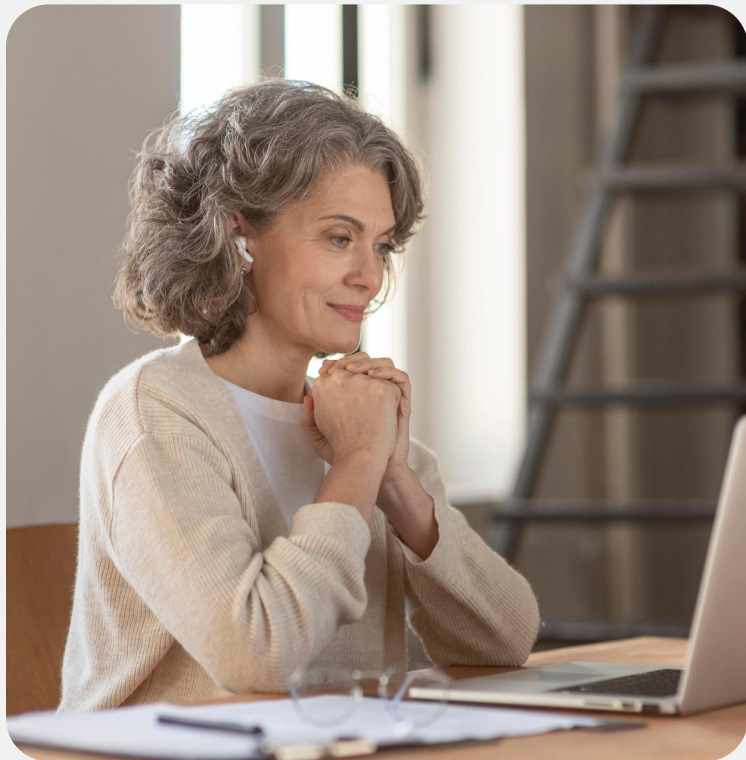
- a. `<img>`
- b. `<image>`
- c. `<pic>`
- d. `<src>`



Ответьте на вопрос

2.3 Какой тег используется для вставки изображения в HTML?

- a. `<img>`
- b. `<image>`
- c. `<pic>`
- d. `<src>`



Ответьте на вопрос

2.4 Какой тег используется для создания таблицы в HTML?

- a. `<tab>`
- b. `<table>`
- c. `<tr>`
- d. `<td>`



Ответьте на вопрос

2.4 Какой тег используется для создания таблицы в HTML?

- a. `<tab>`
- b. `<table>`
- c. `<tr>`
- d. `<td>`



Ответьте на вопрос

2.5 Какой метод BeautifulSoup используется для поиска всех тегов <a> на странице?

- a. `find("a")`
- b. `find_all("a")`
- c. `get_all("a")`
- d. `select("a")`



Ответьте на вопрос

2.5 Какой метод BeautifulSoup используется для поиска всех тегов <a> на странице?

- a. `find("a")`
- b. `find_all("a")`
- c. `get_all("a")`
- d. `select("a")`



Ответьте на вопрос

2.6 Какой метод BeautifulSoup используется для поиска первого тега <h1> на странице?

- a. `find("h1")`
- b. `find_all("h1")`
- c. `get("h1")`
- d. `select("h1")`



Ответьте на вопрос

2.6 Какой метод BeautifulSoup используется для поиска первого тега <h1> на странице?

- a. `find("h1")`
- b. `find_all("h1")`
- c. `get("h1")`
- d. `select("h1")`



Ответьте на вопрос

2.7 Какой метод BeautifulSoup используется для извлечения текста из тега?

- a. `.text()`
- b. `.get_text()`
- c. `.extract()`
- d. `.content()`



Ответьте на вопрос

2.7 Какой метод BeautifulSoup используется для извлечения текста из тега?

- a. `.text()`
- b. `.get_text()`**
- c. `.extract()`
- d. `.content()`



Ответьте на вопрос

2.7 Какой метод BeautifulSoup используется для извлечения текста из тега?

- a. `.text()`
- b. `.get_text()`**
- c. `.extract()`
- d. `.content()`

Выберите верный вариант ответа

2.8 Найдите ошибку в коде:

```
import requests
from bs4 import BeautifulSoup

url = 'https://example.com'
response = requests.get(url)
soup = BeautifulSoup(response.text, 'html.parser')

headings = soup.find_all('h1', 'h2', 'h3')
for heading in headings:
    print(heading.text)
```

- Использовать отдельные вызовы `find_all()` для каждого тега заголовка.
- Использовать метод `find()` вместо `find_all()`.
- Изменить парсер на `'lxml'`.
- Добавить обработку исключений для запроса.

Выберите верный вариант ответа

2.8 Найдите ошибку в коде:

```
import requests
from bs4 import BeautifulSoup

url = 'https://example.com'
response = requests.get(url)
soup = BeautifulSoup(response.text, 'html.parser')

headings = soup.find_all('h1', 'h2', 'h3')
for heading in headings:
    print(heading.text)
```

- Использовать отдельные вызовы `find_all()` для каждого тега заголовка.
- Использовать метод `find()` вместо `find_all()`.
- Изменить парсер на `'lxml'`.
- Добавить обработку исключений для запроса.

Выберите верный вариант ответа

2.9 Найдите ошибку в коде:

```
import requests
from bs4 import BeautifulSoup

url = 'https://example.com'
response = requests.get(url)
soup = BeautifulSoup(response.text, 'html.parser')

table = soup.find('table')
rows = table.find_all('td')
for row in rows:
    print(row.text)
```

- Использовать метод `find_all('tr')` для поиска строк таблицы.
- Использовать метод `find()` вместо `find_all()`.
- Изменить парсер на `'lxml'`.
- Добавить обработку исключений для запроса.

Выберите верный вариант ответа

2.9 Найдите ошибку в коде:

```
import requests
from bs4 import BeautifulSoup

url = 'https://example.com'
response = requests.get(url)
soup = BeautifulSoup(response.text, 'html.parser')

table = soup.find('table')
rows = table.find_all('td')
for row in rows:
    print(row.text)
```

- a. Использовать метод `find_all('tr')` для поиска строк таблицы.
- b. Использовать метод `find()` вместо `find_all()`.
- c. Изменить парсер на 'lxml'.
- d. Добавить обработку исключений для запроса.



ВОПРОСЫ





**Доработайте
практические
задания**



1. Класс User

Создайте класс User, представляющий пользователя некоторого сайта.

- У каждого объекта должно быть имя, фамилия, email, страна и дата рождения.
- Дата передаётся как строка (YYYY-MM-DD) и сохраняется в виде объекта `datetime.date`.
- Определите строковое представление объекта через соответствующий магический метод.
- Добавьте классовое поле `count`, представляющее собой счетчик всех созданных пользователей.
- Добавьте классовый метод `total`, который возвращает текущее количество пользователей.
- Добавьте статический метод `permission(users, age)`. Метод принимает список пользователей и возраст `age` как целое число. Метод возвращает список пользователей, достигших возраста `age` на текущую дату.

Протестируйте работу класса на нескольких примерах.

1. Класс User

Пример для тестирования:

```
u1 = User("Alice", "Smith", "alice@example.com", "USA", "2008-06-15")
u2 = User("Bob", "Johnson", "bob@example.com", "Canada", "1990-11-23")
u3 = User("Carol", "Lee", "carol@example.com", "UK", "2005-01-01")

print(f"Total users: {User.total()}")

print("List of 18+ users:")
for u in User.permission([u1, u2, u3], 18):
    print(u)
```

Пример вывода:

Total users: 3

List of 18+ users:

Bob Johnson (bob@example.com) from Canada, born on 1990-11-23

Carol Lee (carol@example.com) from UK, born on 2005-01-01

2. Использование API

- Воспользуйтесь ресурсом users сайта random-data-api.com. Проверьте его доступность. Запросите данные 10 пользователей. Извлеките из json информацию об имени, фамилии, email, стране и дате рождения.
- Создайте объект Users для каждого полученного пользователя. Создайте список из полученных объектов.
- Выведите данные пользователей, у которых есть доступ к ресурсу с возрастным ограничением 18+.
- Выведите данные пользователей, у которых есть доступ к ресурсу с возрастным ограничением 25+.
- Выведите общее количество пользователей.

Пример вывода:

Пользователи с доступом 18+:

```
Pierre Bahringer (pierre.bahringer@email.com) from United States, born on 1982-05-22
Kent Abernathy (kent.abernathy@email.com) from United States, born on 1972-03-16
Myong Quitzon (myong.quitzon@email.com) from United States, born on 1993-10-04
```

Пользователи с доступом 25+:

```
Pierre Bahringer (pierre.bahringer@email.com) from United States, born on 1982-05-22
Kent Abernathy (kent.abernathy@email.com) from United States, born on 1972-03-16
Myong Quitzon (myong.quitzon@email.com) from United States, born on 1993-10-04
```

Total users: 13



РАЗБОР ДОМАШНЕГО ЗАДАНИЯ



Домашнее задание

1. Предсказание пола по имени

Изучите документацию API предсказания пола по имени: genderize.io/documentation.
Напишите программу, которая запрашивает имя пользователя и выводит его пол и вероятность этого предсказания.

Пример ввода:

Введите имя: Sasha

Пример вывода:

Предполагаемый пол: female

Вероятность: 59.0%

Домашнее задание

1. Получение биржевых данных через API

Напишите программу, которая запрашивает у пользователя тикер (symbol) акций, обращается к функции с ежемесячными данными о результатах торгов (TIME_SERIES_MONTHLY) в API ресурса www.alphavantage.co, запрашивает результат в формате csv, записывает его в файл. Доступность ресурса предварительно нужно проверить. После этого файл требуется прочитать, извлечь данные из столбца volume и вычислить среднее значение этого столбца.

Пример ввода:

Введите тикер: BMWYY

Пример вывода:

CSV-файл успешно сохранён

Средний объём: 7638446.53



ВОПРОСЫ



Домашнее задание

1. Поиск стран

Напишите программу, которая:

- Парсит таблицу с сайта <https://www.iban.com/country-codes> и сохраняет ее в подходящей структуре данных;
- Запрашивает у пользователя код страны (пользователь может ввести двухбуквенный или трехбуквенный код) и выводит название страны, соответствующей этому коду.

Пример ввода:

Введите код страны (2 или 3 буквы): FRA

Пример вывода:

Код FRA соответствует стране: France



ВОПРОСЫ



Заключение

